

TrustLayer

Project Milestone 0 Proposal

CS 415/515
Software Design & Development

Team Members:

Jeremiah Tanner - jtanner3@crimson.ua.edu

Jailin West - jwest2@crimson.ua.edu

Submission Date:

September 2, 2026

Executive Summary:

- TrustLayer is a blockchain backed reputation platform designed to give users with portable and independently verifiable reputation records. Existing online platforms often store reviews and reputation information within centralized systems, meaning users cannot easily transfer or independently verify their reputation outside of a particular platform. TrustLayer addresses this problem by allowing users to establish profiles, create agreements with other users, complete those agreements, and submit reviews based on completed interactions. Verification information associated with these interactions will be recorded on the Solana blockchain, providing an immutable record that can be independently verified. The application will use [Next.js/React](#) frontend, a Rust/Axum backend, PostgreSQL for application data, and Solana for blockchain based verification.

Target Users and Personas

Primary Users:

- TrustLayer's primary users are freelancers, independent contractors, developers, designers, and other individuals who benefit from demonstrating a trustworthy history of completed work.

Secondary Users:

- Secondary users include clients, recruiters, small businesses, and other individuals who want to evaluate the reputation and previous interactions of another user.

Persona 1: Marcus

- Marcus is a 22 year old freelance software developer who works with clients across multiple online platforms. He wants to build a professional reputation that is not controlled by a single platform. His goal is to provide potential clients with verifiable evidence of successful previous interactions.

Persona 2: Sarah

- Sarah owns a small online business and occasionally hires freelance developers and designers. She wants a reliable method for evaluating potential contractors before hiring them. Her primary concern is determining whether reviews and previous work claims are legitimate

Proposed Application

Purpose

- TrustLayer will provide a web based platform where users can establish a portable reputation based on verified interactions with other users.

Major Features

- Account registration and authentication
- Public user profiles
- Solana wallet connection
- Agreements between users
- Agreement completion
- Reviews associated with completed agreements
- Blockchain backed review verification
- Public reputation history

Intended Platform

- TrustLayer will initially be developed as a responsive web application accessible through desktop and mobile web browsers.

Typical User Workflow

User Creates Account -> Creates Profile/Connect Wallet -> Creates Agreement with Another User -> Other User Accepts Agreement -> Work/Interaction Takes Place -> Agreement is Marked Complete -> User Submits Review -> Verification Record Written to Solana -> Review Appears on Public Profile -> Visitors Can Verify Review

Explanation:

For example, a client could create an agreement with a freelance developer through TrustLayer. Once the interaction is completed, the client can submit a review. TrustLayer records verification information associated with the completed interaction on Solana. Future visitors to the developer's profile can then see that the review originated from a verified TrustLayer interaction.

MVP Scope

Included in MVP

- User Registration and login
- User Profiles
- Create agreements
- Complete agreements
- Submits reviews
- Blockchain verification records
- Public reputation profiles
- Verify reputation records

Excluded from MVP

- Cryptocurrency payments
- NFTs
- Messaging
- Mobile application
- AI fraud detection
- Multi-blockchain support
- Advanced dispute resolution

Possible Future Features

- Skill specific reputation
- Reputation badges
- Dispute management
- Advanced reputation scoring
- Fraud detection
- Organization accounts

The MVP will focus on proving TrustLayer's primary concept: two users can complete an interaction, generate a review from that interaction, and create a blockchain backed verification record that can later be independently verified.

Initial Requirements

Priority	User Story
High	As a user, I want to register an account so that I can establish a TrustLayer profile.
High	As a user, I want to create an agreement with another user so the interaction can be verified.
High	As a participant, I want to mark an agreement complete so that it can become eligible for review.
High	As a user, I want to review another user after a completed agreement.
High	As a visitor, I want to view verified reviews on a user's profile.
Medium	As a user, I want to connect my Solana wallet.
Medium	As a visitor, I want to independently verify a reputation record.

Example Acceptance Criteria

Submit Review

- Only participants in a completed agreement may submit a review.
- A review cannot be submitted before the agreement is completed.
- The same participant cannot submit multiple reviews for the same agreement.
- Successful reviews generate blockchain verification information.

Nonfunctional Requirements

Security: Passwords and sensitive credentials must be securely stored and transmitted.

Performance: Normal application requests should return within a reasonable response time under expected course project usage.

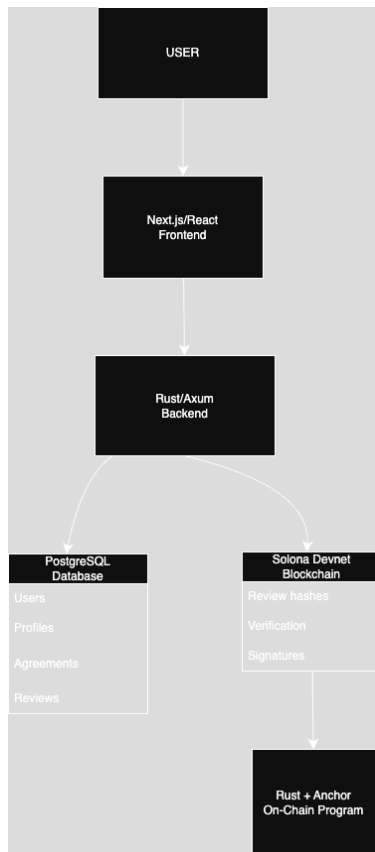
Reliability: Application data must remain consistent between PostgreSQL records and blockchain verification records.

Usability: Major workflows should be understandable without specialized blockchain knowledge.

Accessibility: The web interface should use accessible labels, readable contrast, and keyboard-accessible controls.

Privacy: Personally identifiable information will not be stored directly on the public blockchain.

Initial Design and Architecture



Frontend: [Next.js](#)/React

- Provides the user facing web interface for authentication, profiles, agreements, reviews, and reputation verification.

Backend: Rust/Axum

- Handles application business logic, authentication, database communication, blockchain interactions, and API endpoints. Rust was selected for its performance, memory safety, and relevance to blockchain development.

PostgreSQL

- Stores traditional application data including accounts, profiles, agreements, and review content.

Solana

- Provides the blockchain infrastructure used to store verification information associated with completed interactions and reviews.

Anchor

- Anchor will be evaluated as the framework for developing TrustLayer's Solana on chain program using Rust.

Docker

- Docker will be used to containerize application services and provide a consistent development and deployment environment.

Main Data Entities

User

- id
- email
- password_hash
- wallet_address

Profile

- user_id
- display_name

- bio

Agreement

- id
- creator_id
- participant_id
- title
- description
- status

Review

- id
- agreement_id
- reviewer_id
- reviewed_user_id
- rating
- review_text
- blockchain_transaction

PostgreSQL will store normal application data while Solana will store only information necessary to verify the authenticity of reputation records. Sensitive personal information and full review text will not be stored directly on chain.

External Resources

- Solana Devnet
- Solana RPC services
- Anchor

- PostgreSQL
- Rust/Axum libraries
- Next.js/React libraries
- Docker

Agile Development Process

Sprint Length

- The team will use one week development sprints.

Backlog

- GitHub Issues will be used to track user stories, bugs, technical tasks, and sprint work.

Branching

Use:

main

|

|— feature/authentication

|— feature/agreements

|— feature/reviews

|— feature/blockchain-verification

Development will occur on feature branches. Changes will be submitted through pull requests and reviewed by another team member before being merged into `main`.

Definition of Done

A backlog item is done when:

- Feature is implemented
- Code runs successfully
- Relevant tests pass
- Code has been reviewed
- Documentation is updated when necessary
- Changes are merged through a pull request

Milestone Plan

Milestone 1: Build It

Develop a runnable end-to-end MVP supporting accounts, profiles, agreements, completed interactions, reviews, and initial blockchain verification.

Milestone 2: Test It

Expand the MVP, implement unit and integration testing, perform code reviews, develop a threat model, evaluate relevant OWASP risks, and implement security improvements.

Milestone 3: Deploy It

Containerize TrustLayer, establish CI/CD, deploy the application publicly, complete documentation, create the final v1.0 release, and prepare the live demonstration.

Team Responsibilities

Jeremiah Tanner

- Initial responsibility for backend architecture, Rust/Axum development, blockchain integration, and database integration. Jeremiah will also participate in testing, code review, documentation, deployment, and presentations.

Jailin West

- Initial responsibility for frontend development, UI/UX design, frontend integration, and application testing. Jailin will also participate in code review, documentation, deployment, and presentations.

Contribution Statements

Jeremiah Tanner: Contributed to defining the project concept, MVP scope, technology stack, initial architecture, blockchain integration strategy, and project requirements.

Jailin West: Contributed to defining TrustLayer's user experience, initial feature requirements, product backlog, MVP scope, and overall project planning.

Risk and Mitigation

Risk	Impact	Mitigation
Rust learning curve	Development may take longer	Begin Rust development early and limit backend scope to required features
Blockchain complexity	Solana integration could delay MVP	Build normal application workflow first and integrate blockchain incrementally
Solana/RPC availability	Blockchain requests could fail	Handle RPC failures and test primarily with Devnet
Security vulnerabilities	User accounts/data could be exposed	Follow secure authentication practices and perform OWASP review
Scope creep	Too many features could prevent completion	Keep advanced reputation features outside MVP
Team scheduling	Development could become uneven	Use weekly sprints, GitHub Issues, and regular team communication
Deployment problems	Application may behave differently in production	Use Docker and test deployment before final milestone

Success Criteria

TrustLayer will be considered successful if:

1. Users can register and authenticate successfully.

2. Users can create profiles and agreements with other users.
3. Participants can complete agreements.
4. Users can submit reviews only for valid completed interactions.
5. Review verification information can be successfully recorded on Solana.
6. Visitors can view reputation information and verify blockchain-backed records.
7. The application can be run locally using documented setup instructions.
8. Automated tests verify important application workflows.
9. The final application is containerized and publicly deployed.
10. The team successfully demonstrates the complete workflow from agreement creation through blockchain verification.

References

The Rust Project. *Rust Documentation*. [Rust Documentation](#). Accessed September 1, 2026.

Tokio Contributors. *Axum Documentation*. [Axum Documentation](#). Accessed September 1, 2026.

Solana Foundation. *Solana Documentation*. [Solana Documentation](#). Accessed September 1, 2026.

Anchor. *Anchor Documentation*. [Anchor Documentation](#). Accessed September 1, 2026.

PostgreSQL Global Development Group. *PostgreSQL Documentation*. [PostgreSQL Documentation](#). Accessed September 1, 2026.

Vercel. *Next.js Documentation*. [Next.js Documentation](#). Accessed September 1, 2026.

Docker, Inc. *Docker Documentation*. [Docker Documentation](#). Accessed September 1, 2026.